FuzzyGoal

A C++ Library for Fuzzy-Logic-Based Objective Functions

User Manual

Olaf Frommann

Institute for Aerospace Technology

University of Applied Sciences Bremen

Version: 1.0
Date: June 2026
DOI: DOI will be added after the first GitHub release
Repository: https://github.com/of33/FuzzyGoal

Contents

Revision History

Revision	Date	Changes
1.0	June 2026	Initial public version.

1 Introduction

FuzzyGoal is a small C++ library for constructing and evaluating fuzzy multi-criteria objective functions. It allows users to define several criteria, classify criterion values as desirable, tolerable, or undesirable, combine criteria using fuzzy rules, and compute a scalar objective value.

This manual focuses on practical use of the FuzzyGoal library. It describes installation, compilation, example programs, and the public API. The mathematical background and theoretical motivation are discussed separately in [1].

The manual covers:

- required software and installation,
- compiling and running the example programs,
- basic usage of the FuzzyGoal class,
- creation of criteria and equality constraints,
- creation of fuzzy rules,
- evaluation of single criteria and full objective functions,
- an overview of the public API,
- the provided example programs and their plots.

The main interface of the library is the class FuzzyGoal. A typical workflow is:

1. create a FuzzyGoal object,
2. add one or more criteria,
3. optionally add equality constraints,
4. optionally add fuzzy rules,
5. evaluate the objective function for a given set of criterion values.

The final objective value is a number in the interval $[0, 1]$. Smaller values indicate a more desirable result, while larger values indicate a less desirable result.

2 Requirements and Installation

2.1 Required Software

To compile and run the examples, the following software is required:

- a C++ compiler,
- GNU Make or a compatible `make` implementation,
- `gnuplot` for generating plots.

In addition, `git` is recommended if the repository should be cloned directly from GitHub. Alternatively, the source code can be downloaded as a ZIP archive using a web browser.

All required tools are freely available and can be installed without additional license costs. The installation commands below use commonly available open source software packages.

The library itself consists of the two files:

- `fuzzygoal.h`
- `fuzzygoal.cpp`

The inclusion and compilation of these files are explained in Sections 4 and 8.

2.2 Obtaining the Source Code

There are two common ways to obtain the FuzzyGoal source code:

1. download the repository as a ZIP archive,
2. clone the repository using `git`.

If this manual is read from an already downloaded or cloned repository, this step has already been completed and can be skipped.

Downloading a ZIP Archive

The repository can be downloaded from GitHub using a web browser:

https://github.com/of33/FuzzyGoal

On the GitHub page, select:

`Code` → `Download ZIP`

After downloading, extract the ZIP archive to a directory of your choice. The extracted directory should contain files such as:

```
fuzzygoal.h
fuzzygoal.cpp
Makefile
examples
FuzzyGoal_UserManual.pdf
```

Cloning with Git

Alternatively, the repository can be cloned with `git`:

```
git clone https://github.com/of33/FuzzyGoal.git
cd FuzzyGoal
```

This creates a local copy of the repository and changes into the project directory.

If `git` is not available on your system, install it using the instructions for your operating system below.

2.3 Installing the Required Tools on Windows

On Windows, it is recommended to use MSYS2. MSYS2 provides a modern package manager, Unix-like command line tools, and up-to-date GCC compiler toolchains for building native Windows programs.

Download and install MSYS2 from:

https://www.msys2.org/

Opening the MSYS2 UCRT64 Terminal

After installation, open an MSYS2 terminal. On Windows, this can be done from the Start menu. Search for `MSYS2` and start the entry named `MSYS2 UCRT64`. This terminal provides a suitable environment for compiling native Windows executables with GCC.

MSYS2 uses the package manager `pacman`. It is used to update the system and to install additional software packages.

First update the package database and the core MSYS2 packages:

```
pacman -Syu
```

Here, `-S` means "synchronize packages", `-y` refreshes the package database, and `-u` upgrades installed packages.

If requested, close the terminal window after the update has finished. This is sometimes necessary after core system packages have been updated. Then open `MSYS2 UCRT64` again from the Start menu and continue the update with:

```
pacman -Su
```

This upgrades the remaining installed packages using the already refreshed package database.

Install the compiler, Make, gnuplot, and git:

```
pacman -S --needed git make mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gnuplot
```

The option `-needed` tells `pacman` to skip packages that are already installed and up to date.

The installation can be checked with:

```
g++ --version
make --version
gnuplot --version
git --version
```

If one of these commands is not found, make sure that the `MSYS2 UCRT64` terminal is used. Other MSYS2 terminals, such as the plain `MSYS` terminal, may not provide the same compiler environment by default.

The examples should be built from the `MSYS2 UCRT64` terminal, not from the standard Windows command prompt.

Accessing the FuzzyGoal Directory from MSYS2

After opening the `MSYS2 UCRT64` terminal, the user is inside a Unix-like shell environment. This environment uses Unix-style paths. Windows drives are mounted below `/c`, `/d`, and so on.

For example, the Windows directory

```
C:\Users\Alice\Downloads\FuzzyGoal
```

is accessed in MSYS2 as:

```
/c/Users/Alice/Downloads/FuzzyGoal
```

To enter this directory, run:

```
cd /c/Users/Alice/Downloads/FuzzyGoal
```

The current directory can be displayed with:

```
pwd
```

The files in the current directory can be listed with:

```
ls
```

Note

The MSYS2 terminal uses a Unix-like shell. Commands, file names, directory names, and Makefile targets should be treated as case-sensitive. For example, `make run_basic` is not the same as `make Run_Basic`, and `fuzzygoal.h` is not the same spelling as `FuzzyGoal.h`. Using the exact capitalization shown in this manual is recommended. This also improves portability to Linux and macOS, where file names are commonly case-sensitive.

Before running `make`, check that the current directory contains the project files, for example:

```
fuzzygoal.h
fuzzygoal.cpp
Makefile
examples
```

If the repository has not yet been downloaded, it can also be cloned directly from the `MSYS2 UCRT64` terminal:

```
git clone https://github.com/of33/FuzzyGoal.git
cd FuzzyGoal
```

2.4 Installing the Required Tools on Linux

On Debian-based systems such as Ubuntu, install the required packages with:

```
sudo apt update
sudo apt install g++ make gnuplot git
```

On Fedora, use:

```
sudo dnf install gcc-c++ make gnuplot git
```

On Arch Linux, use:

```
sudo pacman -S gcc make gnuplot git
```

The installation can be checked with:

```
g++ --version
make --version
gnuplot --version
git --version
```

Accessing the FuzzyGoal Directory on Linux

If the repository has not yet been downloaded, it can be cloned with:

```
git clone https://github.com/of33/FuzzyGoal.git
cd FuzzyGoal
```

If the repository was downloaded as a ZIP archive, extract it and change into the extracted directory, for example:

```
cd ~/Downloads/FuzzyGoal
```

Use `pwd` to display the current directory and `ls` to list its contents.

Before running `make`, check that the current directory contains the project files:

```
fuzzygoal.h
fuzzygoal.cpp
Makefile
examples
```

2.5 Installing the Required Tools on macOS

On macOS, install the Xcode command line tools:

```
xcode-select --install
```

This provides a C++ compiler, `make`, `git`, and other basic development tools.

For installing `gnuplot`, Homebrew is recommended. If Homebrew is not installed yet, follow the instructions at:

https://brew.sh/

Then install gnuplot:

```
brew install gnuplot
```

Check the installation with:

```
clang++ --version
make --version
gnuplot --version
git --version
```

Accessing the FuzzyGoal Directory on macOS

If the repository has not yet been downloaded, it can be cloned with:

```
git clone https://github.com/of33/FuzzyGoal.git
cd FuzzyGoal
```

If the repository was downloaded as a ZIP archive, extract it and change into the extracted directory, for example:

```
cd ~/Downloads/FuzzyGoal
```

Use `pwd` to display the current directory and `ls` to list its contents.

Before running `make`, check that the current directory contains the project files:

```
fuzzygoal.h
fuzzygoal.cpp
Makefile
examples
```

3 Quick Start

This section shows the minimal workflow for using `FuzzyGoal` in a C++ program. The example creates one criterion, evaluates the objective function, and checks all returned status codes. For a complete description of all public methods and status codes, see Section 10.

A complete `FuzzyGoal` setup usually follows this pattern:

1. create a `FuzzyGoal` object,
2. add one or more criteria,
3. optionally add rules,
4. call `evaluate` with one value for each registered criterion.

The following example defines a single criterion called `cost`. Smaller values are preferred. The modeled interval is $[0, 100]$, and the reference value is 30.

```
1  #include "fuzzygoal.h"
2
3  #include <iostream>
4
5  int main()
6  {
7      FuzzyGoal goal;
8
```

```
    auto cost = goal.addCriterion(
                    "cost",
                    0.0,                          // c0: lower interval bound
                    100.0,                        // c1: upper interval bound
                    30.0,                         // cRef: reference value
                    FuzzyGoal::SmallerIsBetter,
                    FuzzyGoal::LinearGlobalLinearTol
    );

    if (cost.status != FuzzyGoal::CriterionOk)
    {
        std::cerr << "Could not create criterion: "
                  << FuzzyGoal::statusToString(cost.status)
                  << "\n";
        return 1;
    }

    double objective = 0.0;

    auto status = goal.evaluate(
        {
            {cost.id, 40.0}
        },
        objective
    );

    if (status != FuzzyGoal::EvaluationOk)
    {
        std::cerr << "Evaluation failed: "
                  << FuzzyGoal::statusToString(status)
                  << "\n";
        return 1;
    }

    std::cout << "Objective value = " << objective << "\n";

    return 0;
}
```

The program can be compiled manually with:

```
g++ main.cpp fuzzygoal.cpp -std=c++11 -O2 -Wall -Wextra -pedantic -I. -o main
```

On Linux and macOS, run it with:

```
./main
```

On Windows, the executable may be called:

```
main.exe
```

The computed objective value lies in the interval $[0, 1]$. Smaller values indicate a more desirable result.

Note

The examples use the C++ keyword `auto` in several places. In modern C++, `auto` tells the compiler to deduce the variable type from the expression on the right-hand side. For example,

```
auto baseC1 = fuzzyBase.addCriterion(...);
auto rule   = fuzzyRule.addRule(...);
auto status = fuzzyBase.evaluate(...);
```

is equivalent to writing the explicit types:

```
FuzzyGoal::NewCriterion baseC1 = fuzzyBase.addCriterion(...);
FuzzyGoal::NewRule rule        = fuzzyRule.addRule(...);
FuzzyGoal::EvaluationStatus status = fuzzyBase.evaluate(...);
```

Using `auto` makes the examples shorter, but it is not required. Users who prefer explicit types may write them out.

Adding a Rule

Rules are optional. They can be used to express additional preferences between criterion memberships.

For example, if two criteria have already been created, a rule can be added as follows:

```
1  auto rule = goal.addRule(
2                  criterionA.id, FuzzyGoal::Undesirable,
3                  criterionB.id, FuzzyGoal::Undesirable,
4                  FuzzyGoal::Or,
5                  FuzzyGoal::Undesirable
6  );
7
8  if (rule.status != FuzzyGoal::RuleOk)
9  {
10     std::cerr << "Could not create rule: "
11               << FuzzyGoal::statusToString(rule.status)
12               << "\n";
13     return 1;
14 }
```

This rule can be read as:

> If criterion A is undesirable or criterion B is undesirable, then the overall result receives an additional undesirable contribution.

The ID stored in `rule.id` can be used later if the rule should be removed with `removeRule`.

4 Basic Usage

This section introduces the basic usage of the library using the first example program. The example defines a simple one-dimensional test problem with two conflicting criteria,

$$c_1(x) = x, \qquad c_2(x) = 1 - x, \qquad x \in [0, 1].$$

Both criteria are minimized. Therefore, c_1 is best near $x = 0$, while c_2 is best near $x = 1$. This creates a simple conflict between two objectives.

The example compares two fuzzy objective functions:

1. a base objective using only the implicit criterion classification,
2. a rule-based objective using the same criteria plus one explicit fuzzy rule.

The program writes its results to:

```
basic_fuzzy_example.dat
```

4.1 Including the Library

The library is included with:

```
1 #include "fuzzygoal.h"
```

The file `fuzzygoal.cpp` must be compiled together with the example program.

4.2 Defining the Criterion Interval

In the first example, both criteria use the same interval and reference value:

```
1 const double c0   = 0.0;
2 const double c1   = 1.0;
3 const double cref = 0.5;
```

Thus, the modeled interval is $[0, 1]$, and the reference value is placed at the center of the interval.

The membership function variant used in this example is:

```
1 FuzzyGoal::LinearRefLinearTol
```

4.3 Creating a Base Objective

The first objective uses only the implicit single-criterion classification.

```
1 FuzzyGoal fuzzyBase;
```

Two criteria are added:

```
1  auto baseC1 = fuzzyBase.addCriterion(
2                  "c1",
3                  c0, c1, cref,
4                  FuzzyGoal::SmallerIsBetter,
5                  FuzzyGoal::LinearRefLinearTol
6  );
7
8  auto baseC2 = fuzzyBase.addCriterion(
9                  "c2",
10                 c0, c1, cref,
```

```
                FuzzyGoal::SmallerIsBetter,
                FuzzyGoal::LinearRefLinearTol
);
```

Both criteria use:

```
FuzzyGoal::SmallerIsBetter
```

because both c_1 and c_2 are minimized.

The returned status should always be checked:

```
if (baseC1.status != FuzzyGoal::CriterionOk ||
    baseC2.status != FuzzyGoal::CriterionOk)
{
    std::cerr << "Failed to create criteria for base objective:\n"
              << "  c1: " << FuzzyGoal::statusToString(baseC1.status) << "\n"
              << "  c2: " << FuzzyGoal::statusToString(baseC2.status) << "\n";
    return 1;
}
```

The criterion IDs stored in `baseC1.id` and `baseC2.id` are used later for evaluation.

4.4 Creating a Rule-Based Objective

The second objective uses the same two criteria but adds one explicit fuzzy rule.

```
FuzzyGoal fuzzyRule;
```

The criteria are created in the same way:

```
auto ruleC1 = fuzzyRule.addCriterion(
                "c1",
                c0, c1, cref,
                FuzzyGoal::SmallerIsBetter,
                FuzzyGoal::LinearRefLinearTol
);

auto ruleC2 = fuzzyRule.addCriterion(
                "c2",
                c0, c1, cref,
                FuzzyGoal::SmallerIsBetter,
                FuzzyGoal::LinearRefLinearTol
);
```

The explicit rule is added with `addRule`:

```
auto rule = fuzzyRule.addRule(
                ruleC1.id, FuzzyGoal::Undesirable,
                ruleC2.id, FuzzyGoal::Undesirable,
                FuzzyGoal::Or,
                FuzzyGoal::Undesirable
);
```

This rule can be read as:

> If c_1 is undesirable or c_2 is undesirable, then the overall result receives an additional undesirable contribution.

The rule penalizes extreme solutions. At $x = 0$, criterion $c_2 = 1 - x$ is large and therefore undesirable. At $x = 1$, criterion $c_1 = x$ is large and therefore undesirable. The rule therefore favors a compromise near the center.

The return value of `addRule` contains a status code and, if rule creation was successful, the ID of the new rule:

```
if (rule.status != FuzzyGoal::RuleOk)
{
    std::cerr << "Failed to add fuzzy rule: "
              << FuzzyGoal::statusToString(rule.status) << "\n";
    return 1;
}
```

4.5 Evaluating Criterion Memberships

For each sample point x, the example computes the numerical criterion values that are passed to the `FuzzyGoal` object:

```
const double valueC1 = x;
const double valueC2 = 1.0 - x;
```

Thus, `valueC1` $= c_1(x) = x$, `valueC2` $= c_2(x) = 1 - x$.

The variables `valueC1` and `valueC2` are used both for evaluating the individual memberships and for evaluating the full objective function.

In typical applications, the individual membership values are not needed explicitly. They are computed internally when `evaluate` is called. The method `evaluateCriterionMembership` is mainly useful for debugging, inspection, and plotting. In this example, it is used to write the membership values to the output file.

The memberships of individual criteria are computed with `evaluateCriterionMembership`:

```
FuzzyGoal::CriterionMembership m1;
FuzzyGoal::CriterionMembership m2;

auto mStatus1 = fuzzyBase.evaluateCriterionMembership(baseC1.id, valueC1, m1);

auto mStatus2 = fuzzyBase.evaluateCriterionMembership(baseC2.id, valueC2, m2);

if (mStatus1 != FuzzyGoal::EvaluationOk ||
    mStatus2 != FuzzyGoal::EvaluationOk)
{
    std::cerr << "Membership evaluation failed:\n"
              << "  c1: " << FuzzyGoal::statusToString(mStatus1) << "\n"
              << "  c2: " << FuzzyGoal::statusToString(mStatus2) << "\n";
    return 1;
}
```

The result is stored in objects of type `FuzzyGoal::CriterionMembership`. Each object contains:

```
double desirable;
double tolerable;
double undesirable;
```

4.6 Evaluating the Full Objective

The full fuzzy objective is evaluated with `evaluate`. For the base objective, the call is:

```
double fBase = 0.0;

auto baseStatus = fuzzyBase.evaluate(
                    {{baseC1.id, valueC1},
                     {baseC2.id, valueC2}},
                    fBase
);

if (baseStatus != FuzzyGoal::EvaluationOk)
{
    std::cerr << "Base objective evaluation failed: "
              << FuzzyGoal::statusToString(baseStatus) << "\n";
    return 1;
}
```

For the rule-based objective:

```
double fRule = 0.0;

auto ruleStatus = fuzzyRule.evaluate(
                    {{ruleC1.id, valueC1},
                     {ruleC2.id, valueC2}},
                    fRule
);

if (ruleStatus != FuzzyGoal::EvaluationOk)
{
    std::cerr << "Rule objective evaluation failed: "
              << FuzzyGoal::statusToString(ruleStatus) << "\n";
    return 1;
}
```

The input to `evaluate` is a list of pairs. Each pair contains a criterion ID and the current value of that criterion.

The return value is an `EvaluationStatus`. If the returned status is `EvaluationOk`, the scalar objective value has been written to the output variable. Otherwise, the status code describes why evaluation failed.

The evaluation is strict:

- every registered criterion must be provided exactly once,
- unknown criterion IDs are not allowed,
- missing criterion values cause the evaluation to fail,
- duplicate criterion values cause the evaluation to fail.

4.7 Output File

The first example writes the file:

```
basic_fuzzy_example.dat
```

The columns are:

Column	Description
1	x
2	$c_1 = x$
3	$c_2 = 1 - x$
4	desirable membership of c_1
5	tolerable membership of c_1
6	undesirable membership of c_1
7	desirable membership of c_2
8	tolerable membership of c_2
9	undesirable membership of c_2
10	objective value without explicit rule
11	objective value with explicit rule

Without the explicit rule, the base objective `f_base` is constant. This is a consequence of the symmetry of the example. The two criteria are $c_1(x) = x, \quad c_2(x) = 1 - x$, and both are minimized using the same membership function. Therefore, an improvement of one criterion is exactly balanced by a deterioration of the other criterion. Since no additional preference rule is used in the base case, the defuzzified result remains at the neutral value `f_base` $= 0.5$ for all sampled values of x.

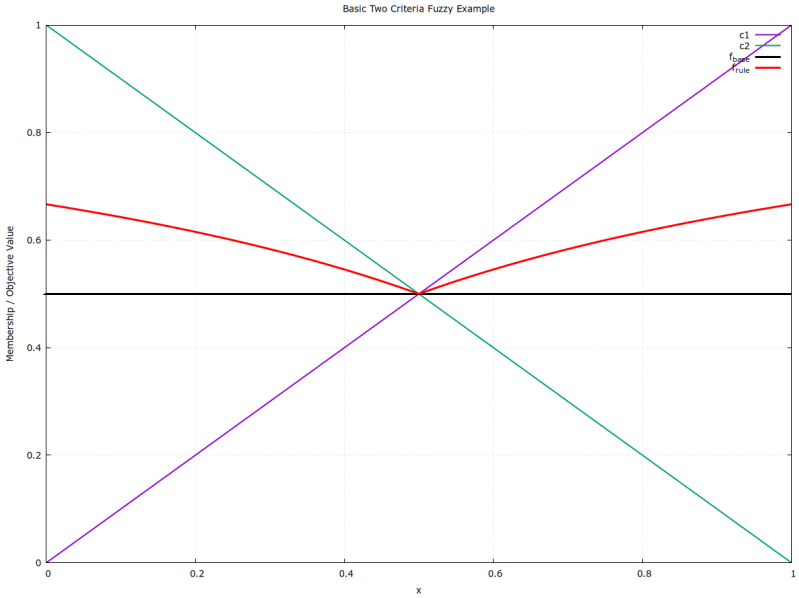

Figure 1: Output of the first example. The plot compares the two criteria $c_1 = x$ and $c_2 = 1 - x$, the base objective `f_base`, and the rule-based objective `f_rule`. Without the explicit rule, the two opposing criteria balance each other exactly, resulting in the constant neutral value `f_base` $= 0.5$. The additional rule penalizes undesirable extreme solutions and therefore changes the shape of the objective.

5 Example Programs

The distribution contains three example programs. They demonstrate typical use cases of the library and can be used as templates for new applications. Each example can be built, executed, and plotted using a dedicated Makefile target. The targets are listed in the corresponding subsections.

5.1 Example 1: Basic Two-Criteria Fuzzy Objective

The first example defines two conflicting criteria:

$c_1(x) = x, \qquad c_2(x) = 1 - x.$

Both are minimized. The example compares a base objective without explicit rules to a rule-based objective with the rule:

> If c_1 is undesirable or c_2 is undesirable, then the result is undesirable.

The example writes:

```
basic_fuzzy_example.dat
```

The corresponding plot script is:

```
plot_01_basic.gp
```

It generates:

```
01_basic_fuzzy_example.png
```

To build, run, and plot this example using the provided Makefile, execute:

```
make run_basic
```

This command runs the executable, writes the data file, opens the gnuplot window, and saves the plot as a PNG file after the user confirms the prompt.

5.2 Example 2: Two-Dimensional Topography and Rule Comparison

The second example evaluates fuzzy objectives on a two-dimensional parameter domain. The parameters are Λ and $\bar{t}$, sampled on a grid (see [1], extended testcase 1).

The program computes two normalized criteria, denoted c_1 and c_2. It then compares three different fuzzy rule formulations using three independent `FuzzyGoal` objects.

The membership function used in this example is:

```
1 FuzzyGoal::ParabolicGlobalParabolicTol
```

The boundary behavior is:

```
1 FuzzyGoal::LinearGuidance
```

The rule aggregation is:

```
1 FuzzyGoal::Aggregation::PNormAgg(-30.0)
```

The three rules are:

1. If c_1 is desirable and c_2 is desirable, then the result is desirable.
2. If c_1 is desirable and c_2 is tolerable, then the result is tolerable.
3. If c_1 is desirable or c_2 is undesirable, then the result is undesirable.

The third rule is intentionally a problematic preference formulation. It is included to illustrate how a poorly chosen rule can affect the resulting objective landscape.

The example writes:

```
c1_c2_topography_fuzzy.dat
```

The columns are:

Column	Description
1	Λ
2	$\bar{t}$
3	c_1
4	c_2
5	objective value for rule 1
6	objective value for rule 2
7	objective value for rule 3

The plot script is:

```
plot_02_topography.gp
```

It is called with a column number and labels. For example:

```
gnuplot -c plot_02_topography.gp 5 "Rule1" "Desirable AND Desirable"
gnuplot -c plot_02_topography.gp 6 "Rule2" "Desirable AND Tolerable"
gnuplot -c plot_02_topography.gp 7 "Rule3" "Desirable OR Undesirable"
```

The generated plot files are:

```
c1_c2_topography_fuzzy_rule1.png
c1_c2_topography_fuzzy_rule2.png
c1_c2_topography_fuzzy_rule3.png
```

To build, run, and plot this example using the provided Makefile, execute:

```
make run_topography
```

This command runs the topography example and calls the gnuplot script three times, once for each rule.

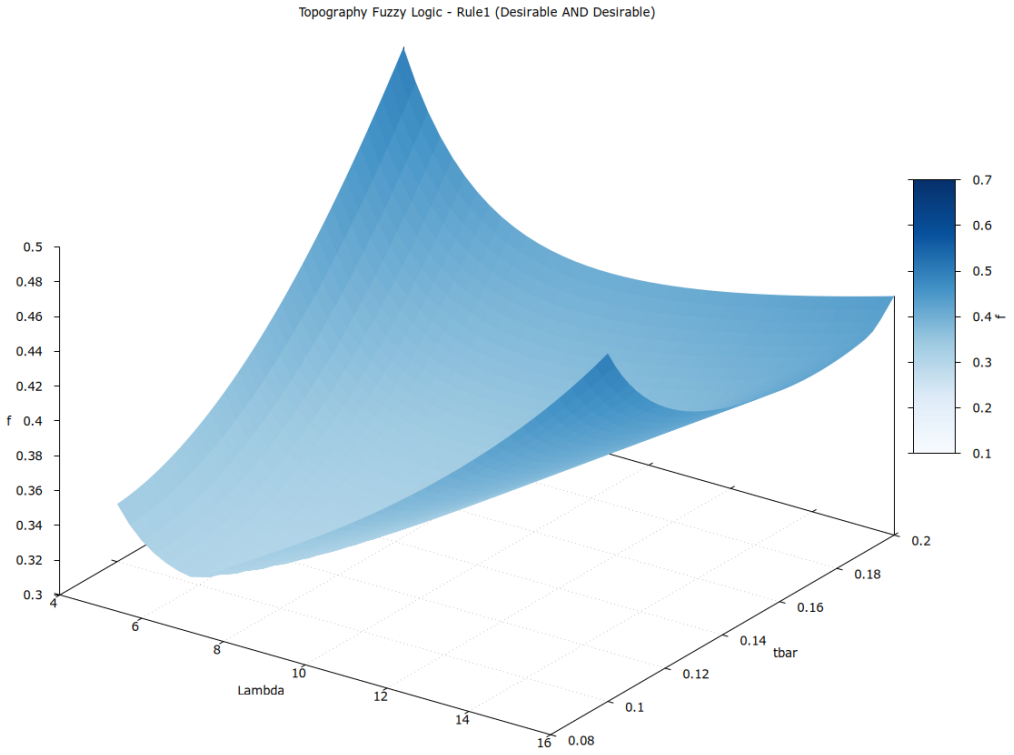

Figure 2: Topography for rule 1: desirable and desirable implies desirable.

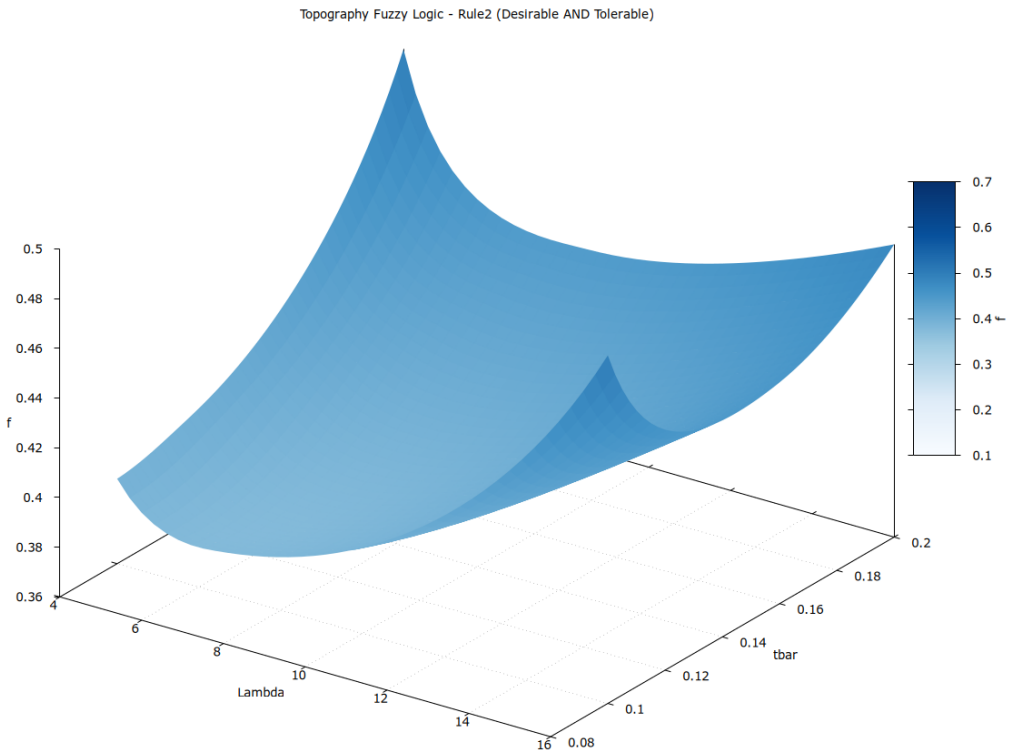

Figure 3: Topography for rule 2: desirable and tolerable implies tolerable.

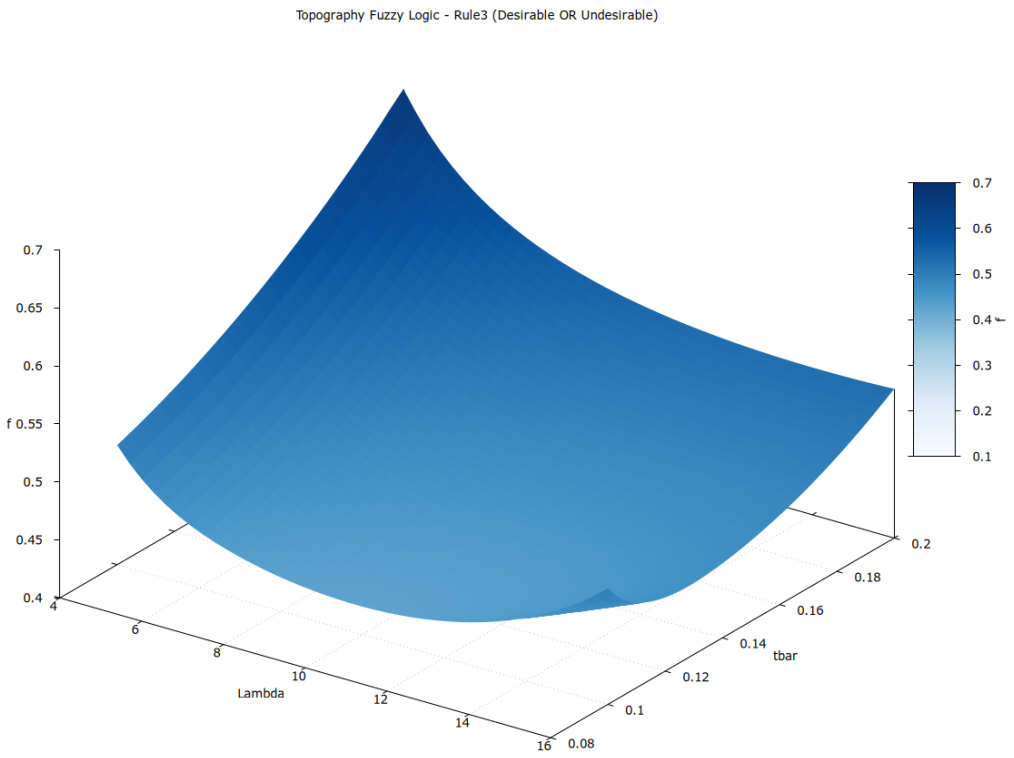

Figure 4: Topography for rule 3: desirable or undesirable implies undesirable. This rule is intentionally problematic.

5.3 Example 3: Equality Constraints

The third example demonstrates the use of equality constraints.

The equality condition is:

$g(x) = x, \qquad g^* = 0.5.$

Internally, the library evaluates the absolute deviation:

$c = |x - 0.5|.$

The example uses:

```
const double target    = 0.5;
const double tolerance = 0.05;
const double gamma     = 10.0;
```

The equality constraint is created with:

```
auto eq = equalityOnly.addEqualityConstraint(
            "x_equals_0.5",
            target,
            tolerance,
            FuzzyGoal::GaussianGlobalGaussianTol,
            0.15,
            gamma
);
```

The example compares two cases:

1. an objective containing only the equality constraint,
2. an objective combining the equality constraint with an additional inequality-like criterion.

The additional criterion models the preference $x \leq 0.5$ by using `SmallerIsBetter` and the reference value 0.5:

```
auto ineq = equalityPlusInequality.addCriterion(
                "x_less_equal_0.5",
                0.0,
                1.0,
                target,
                FuzzyGoal::SmallerIsBetter,
                FuzzyGoal::LinearGlobalParabolicTol,
                FuzzyGoal::LinearGuidance
);
```

The example writes:

```
equality_constraint_example.dat
```

The columns are:

Column	Description
1	x
2	objective value for equality-only case
3	objective value for equality plus inequality-like criterion

The plot script is:

```
plot_03_equality.gp
```

It generates:

```
03_equality_constraint_example.png
```

To build, run, and plot this example using the provided Makefile, execute:

```
make run_equality
```

This command runs the equality constraint example and generates the corresponding plot.

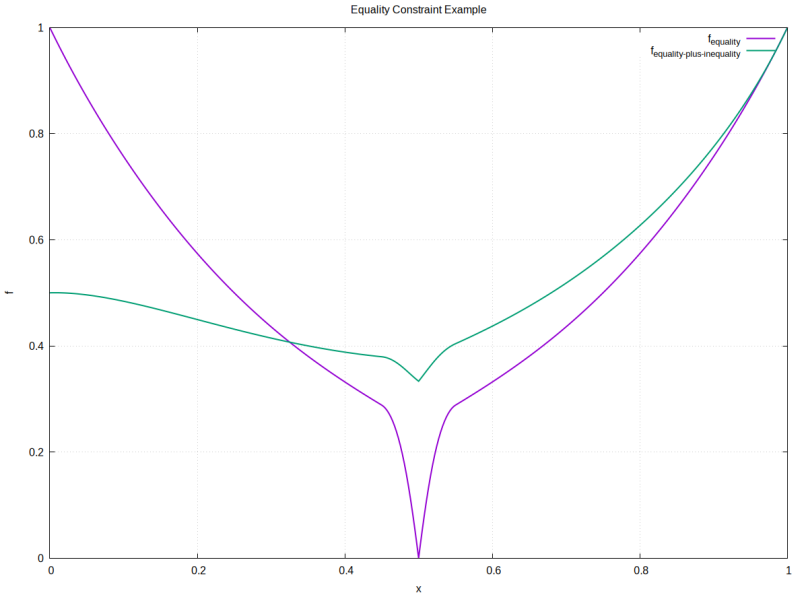

Figure 5: Equality constraint example. The equality-only objective is compared to a combination of equality and inequality-like preference.

6 Building and Running the Examples

The examples are built using the provided Makefile. The Makefile expects the following project structure:

```
fuzzygoal.h
fuzzygoal.cpp
Makefile
examples/01_basic_two_criteria.cpp
examples/02_extended_testcase1_topography.cpp
examples/03_equality_constraint.cpp
plot_01_basic.gp
plot_02_topography.gp
plot_03_equality.gp
```

The exact names of the example source files must match the files in the `examples` directory, because the Makefile automatically builds all `.cpp` files in that directory.

6.1 Running the Test Suite

The repository contains a small test suite that checks criterion creation, rule creation, evaluation status codes, equality constraints, and a regression case.

The tests can be built and run with:

```
make test
```

A successful test run ends with:

```
All tests passed.
```

6.2 Building All Examples

To build all examples, run:

```
make
```

or explicitly:

```
make all
```

The Makefile creates two directories:

```
build/
bin/
```

Object files are placed in `build/`. Executables are placed in `bin/`.

6.3 Running the Examples

The first example can be run and plotted with:

```
make run_basic
```

This executes the first example and then calls:

```
gnuplot plot_01_basic.gp
```

The second example can be run and plotted with:

```
make run_topography
```

This runs the topography example and calls the plot script three times, once for each rule.

The third example can be run and plotted with:

```
make run_equality
```

This runs the equality constraint example and then calls:

```
gnuplot plot_03_equality.gp
```

All example executables can be run without plotting using:

```
make run
```

6.4 Cleaning the Directory

Generated build files, data files, and PNG images can be removed with:

```
make clean
```

This removes:

- the `build/` directory,
- the `bin/` directory,
- generated `.dat` files,
- generated `.png` files.

6.5 Help Target

The Makefile provides a help target:

```
make help
```

It prints a short overview of the available build and run commands.

7 Plot Generation with Gnuplot

The example programs write data files. The corresponding gnuplot scripts read these files and generate PNG images.

The scripts first open an interactive plot window. After pressing the return key in the console, the plot is saved as a PNG file.

7.1 Basic Example Plot

The first plot script is:

```
plot_01_basic.gp
```

It reads:

```
basic_fuzzy_example.dat
```

and writes:

```
01_basic_fuzzy_example.png
```

7.2 Topography Plots

The second plot script is parameterized:

```
gnuplot -c plot_02_topography.gp <column> "<rule>" "<description>"
```

For the provided example, the intended calls are:

```
gnuplot -c plot_02_topography.gp 5 "Rule1" "Desirable AND Desirable"
gnuplot -c plot_02_topography.gp 6 "Rule2" "Desirable AND Tolerable"
gnuplot -c plot_02_topography.gp 7 "Rule3" "Desirable OR Undesirable"
```

The output files are:

```
c1_c2_topography_fuzzy_rule1.png
c1_c2_topography_fuzzy_rule2.png
c1_c2_topography_fuzzy_rule3.png
```

7.3 Equality Constraint Plot

The third plot script is:

```
plot_03_equality.gp
```

It reads:

```
equality_constraint_example.dat
```

and writes:

```
03_equality_constraint_example.png
```

8 Manual Compilation without Makefile

A user program can also be compiled manually. For example, a file `main.cpp` using the library can be compiled with:

```
g++ main.cpp fuzzygoal.cpp -std=c++11 -O2 -Wall -Wextra -pedantic -I. -o main
```

On Linux and macOS, the executable can then be run with:

```
./main
```

On Windows, the executable may be called:

```
main.exe
```

9 Troubleshooting

This section lists common problems and possible solutions. If the examples fail unexpectedly, run `make test` to check the core library functionality.

9.1 Criterion Creation Fails

Criterion creation methods return a `FuzzyGoal::NewCriterion` object. If the member `status` is not `CriterionOk`, the criterion was not created and the returned ID is invalid.

Example:

```
auto c = goal.addCriterion(...);

if (c.status != FuzzyGoal::CriterionOk)
{
    std::cerr << FuzzyGoal::statusToString(c.status) << "\n";
}
```

Common causes are:

- `c1` is not larger than `c0`,
- `cRef` is outside the interval,
- `cRef` is too close to one of the interval boundaries,
- a Gaussian membership function was selected but `sigmaRel` is not positive.

For ordinary criteria, make sure that: $c_0 < c_{\text{Ref}} < c_1$.

9.2 `RuleUnknownCriterionA` or `RuleUnknownCriterionB`

These status codes indicate that a rule refers to a criterion ID that does not exist in the corresponding `FuzzyGoal` object.

Typical causes are:

- the criterion was not created successfully,
- the wrong criterion ID was used,
- the criterion ID belongs to another `FuzzyGoal` object,
- the criterion was removed before the rule was added.

Example of correct usage:

```
auto c1 = goal.addCriterion(...);
auto c2 = goal.addCriterion(...);

if (c1.status != FuzzyGoal::CriterionOk ||
    c2.status != FuzzyGoal::CriterionOk)
{
    std::cerr << "Criterion creation failed.\n";
    return 1;
}

auto rule = goal.addRule(
                c1.id, FuzzyGoal::Desirable,
                c2.id, FuzzyGoal::Desirable,
                FuzzyGoal::And,
                FuzzyGoal::Desirable
);
```

Criterion IDs should not be mixed between different `FuzzyGoal` objects.

9.3 RuleInvalidAggregation

This status code indicates that the selected aggregation descriptor has invalid parameters.

Check the following conditions:

- for `SoftMinMax`, the parameter must be positive,
- for `PNorm`, the parameter must be negative.

Correct examples:

```
auto soft = FuzzyGoal::Aggregation::SoftMinMaxAgg(1e-4);
auto pnorm = FuzzyGoal::Aggregation::PNormAgg(-30.0);
```

Incorrect examples:

```
auto soft = FuzzyGoal::Aggregation::SoftMinMaxAgg(0.0);
auto pnorm = FuzzyGoal::Aggregation::PNormAgg(2.0);
```

9.4 EvaluationMissingCriterion

The method `evaluate` requires exactly one value for every registered criterion. This status code means that at least one criterion was not provided.

For example, if two criteria were created, both must be passed to `evaluate`:

```
auto status = goal.evaluate(
    {
        {c1.id, value1},
        {c2.id, value2}
    },
    objective
);
```

Providing only one of them causes evaluation to fail:

```
auto status = goal.evaluate(
    {
        {c1.id, value1}
    },
    objective
);
```

9.5 EvaluationDuplicateCriterion

This status code means that the same criterion ID was provided more than once during objective evaluation.

Incorrect example:

```
auto status = goal.evaluate(
    {
        {c1.id, value1},
        {c1.id, value2}
    },
    objective
);
```

Each registered criterion must occur exactly once.

9.6 EvaluationUnknownCriterion

This status code means that the input vector passed to `evaluate` contains an ID that does not refer to a registered criterion.

Typical causes are:

- a wrong or uninitialized ID was used,
- an ID from another `FuzzyGoal` object was used,
- the criterion was removed before evaluation.

Check that all IDs in the input vector were returned by successful calls to `addCriterion` or `addEqualityConstraint` on the same `FuzzyGoal` object.

9.7 EvaluationDefuzzificationFailed

This status code indicates that the inference result could not be converted into a scalar objective value. In normal usage this should rarely occur.

Possible causes are:

- no effective membership contribution was available,
- the objective was evaluated in an inconsistent internal state.

Check that at least one criterion is registered and that the input vector is complete and valid.

9.8 Gnuplot Is Not Found

If the Makefile prints a message such as:

```
Gnuplot not found. Please install as described in the documentation.
```

then `gnuplot` is either not installed or not available in the current terminal environment.

Check the installation with:

```
gnuplot --version
```

If this command fails, install `gnuplot` as described in Section 2.

On Windows, make sure that the `MSYS2 UCRT64` terminal is used.

9.9 make Is Not Found

If the command

```
make
```

is not found, the build tool is either not installed or not available in the current terminal environment.

On Linux, install `make` using the package manager of the distribution. On macOS, install the Xcode command line tools. On Windows, use the `MSYS2 UCRT64` terminal and install the required packages as described in Section 2.

Check the installation with:

```
make --version
```

9.10 The Plot Window Opens but No PNG File Is Written

The provided gnuplot scripts first open an interactive plot window. The PNG file is written only after the user confirms the prompt in the console.

When the message

```
Press [ENTER] inside console to save picture as PNG and close program ...
```

appears, press the return key in the terminal window from which gnuplot was started.

9.11 The Program Compiles but Evaluation Always Fails

If compilation succeeds but `evaluate` returns an error status, check the following points:

- Were all criteria created successfully?
- Are all criterion IDs taken from the same `FuzzyGoal` object?
- Does the input vector contain exactly one value for every registered criterion?
- Are there duplicated criterion IDs?
- Were any criteria removed before evaluation?

For debugging, print the returned status:

```
1 auto status = goal.evaluate(values, objective);
2
3 if (status != FuzzyGoal::EvaluationOk)
4 {
5     std::cerr << "Evaluation failed: "
6               << FuzzyGoal::statusToString(status)
7               << "\n";
8 }
```

10 Public API Reference

This section describes the public interface of the library. Internal helper classes and implementation details are intentionally omitted.

The public API is centered around the class `FuzzyGoal`. A `FuzzyGoal` object stores criteria, equality constraints, fuzzy rules, and provides methods for evaluating individual memberships and full scalar objective values.

10.1 Public Interface Overview

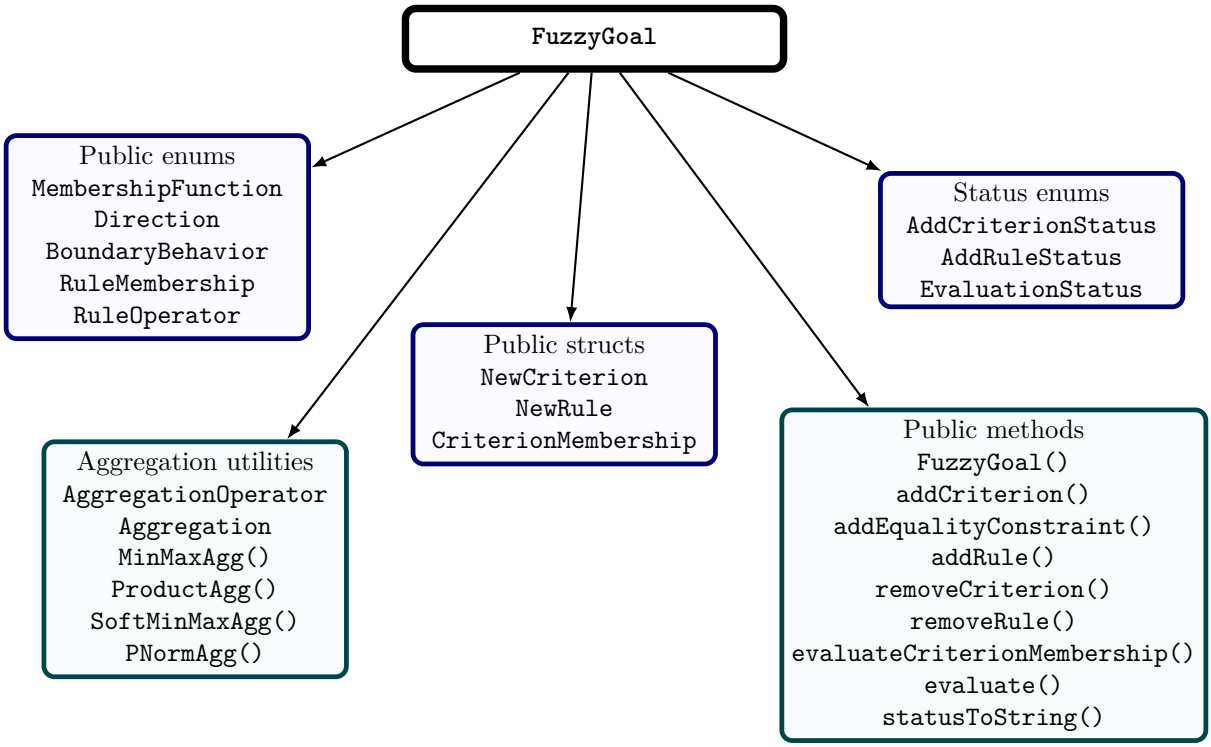

Figure 6: Overview of the public `FuzzyGoal` interface.

10.2 FuzzyGoal

class FuzzyGoal

Header

`fuzzygoal.h`

Description

`FuzzyGoal` is the main class of the library. A `FuzzyGoal` object represents one fuzzy objective function. It stores all criteria, equality constraints, and explicit fuzzy rules belonging to this objective.

A typical usage pattern is:

1. create a `FuzzyGoal` object,
2. add criteria or equality constraints,
3. optionally add fuzzy rules,
4. evaluate criterion memberships or the complete objective function.

Example

```
FuzzyGoal goal;
```

10.3 Public Types

10.3.1 FuzzyGoal::MembershipFunction

enum FuzzyGoal::MembershipFunction

Description

Selects the membership function variant used by a criterion. Each variant defines how the desirable, tolerable, and undesirable memberships are computed.

Value	Description
`LinearRefLinearTol`	Linear desirable and undesirable memberships with respect to the reference value; linear tolerable membership.
`LinearRefParabolicTol`	Linear desirable and undesirable memberships with respect to the reference value; parabolic tolerable membership.
`LinearGlobalLinearTol`	Linear desirable and undesirable memberships over the global interval; linear tolerable membership.
`LinearGlobalParabolicTol`	Linear desirable and undesirable memberships over the global interval; parabolic tolerable membership.
`ParabolicRefParabolicTol`	Parabolic desirable and undesirable memberships with respect to the reference value; parabolic tolerable membership.
`ParabolicGlobalParabolicTol`	Parabolic desirable and undesirable memberships over the global interval; parabolic tolerable membership.
`LinearGlobalGaussianTol`	Linear global desirable and undesirable memberships; Gaussian tolerable membership.
`GaussianGlobalGaussianTol`	Gaussian global desirable and undesirable memberships; Gaussian tolerable membership.

Notes

Variants containing `Gaussian` require a positive `sigmaRel` value. If no value is specified explicitly, the convenience overloads use `sigmaRel = 0.15`.

10.3.2 FuzzyGoal::Direction

enum FuzzyGoal::Direction

Description

Specifies whether smaller or larger criterion values are preferred.

Value	Description
SmallerIsBetter	Smaller criterion values are preferred. Typical examples are cost, error, mass, runtime, or energy consumption.
LargerIsBetter	Larger criterion values are preferred. Typical examples are accuracy, efficiency, lifetime, or throughput.

10.3.3 FuzzyGoal::BoundaryBehavior

enum FuzzyGoal::BoundaryBehavior

Description

Specifies how criterion values outside the modeled interval are handled.

Value	Description
HardSaturation	Values outside the interval are saturated. On the favorable side, the desirable membership becomes one. On the unfavorable side, the undesirable membership becomes one.
LinearGuidance	On the favorable side outside the interval, the desirable membership is reduced linearly. This avoids a flat saturated region and can provide additional guidance in optimization settings.

10.3.4 FuzzyGoal::AddCriterionStatus

enum FuzzyGoal::AddCriterionStatus

Description

Reports whether a criterion or equality constraint was created successfully.

Value	Description
`CriterionOk`	The criterion or equality constraint was created successfully.
`InvalidInterval`	The interval is invalid. For ordinary criteria, this usually means `c1 <= c0`. For equality constraints, this can also indicate invalid `tolerance` or `gamma`.
`RefBelowInterval`	The reference value is below the interval.
`RefAboveInterval`	The reference value is above the interval.
`RefTooCloseToLowerBound`	The reference value is too close to the lower interval boundary.
`RefTooCloseToUpperBound`	The reference value is too close to the upper interval boundary.
`InvalidSigma`	A Gaussian membership function was selected, but `sigmaRel` is not positive.

10.3.5 FuzzyGoal::AddRuleStatus

enum FuzzyGoal::AddRuleStatus

Description

Reports whether a fuzzy rule was created successfully.

Value	Description
`RuleOk`	The rule was created successfully.
`RuleUnknownCriterionA`	The first criterion ID does not refer to an existing criterion.
`RuleUnknownCriterionB`	The second criterion ID does not refer to an existing criterion.
`RuleInvalidAggregation`	The selected aggregation descriptor has invalid parameters. For `SoftMinMax`, the parameter must be positive. For `PNorm`, the parameter must be negative.

10.3.6 FuzzyGoal::EvaluationStatus

enum FuzzyGoal::EvaluationStatus

Description

Reports whether a membership or objective evaluation was successful.

Value	Description
EvaluationOk	The evaluation was successful.
EvaluationUnknownCriterion	An unknown criterion ID was used.
EvaluationMissingCriterion	At least one registered criterion was not provided during objective evaluation.
EvaluationDuplicateCriterion	At least one criterion was provided more than once during objective evaluation.
EvaluationDefuzzificationFailed	The inference result could not be defuzzified. This usually indicates that no effective membership contribution was available.

10.3.7 FuzzyGoal::NewCriterion

struct FuzzyGoal::NewCriterion

Description

Return type used by methods that create criteria or equality constraints.

Member	Description
AddCriterionStatus status	Status code describing whether creation was successful.
int id	Identifier of the created criterion. Valid only if status == FuzzyGoal::CriterionOk.

10.3.8 FuzzyGoal::NewRule

struct FuzzyGoal::NewRule

Description

Return type used by methods that create fuzzy rules.

Member	Description
AddRuleStatus status	Status code describing whether rule creation was successful.
int id	Identifier of the created rule. Valid only if status == FuzzyGoal::RuleOk.

10.3.9 `FuzzyGoal::CriterionMembership`

struct FuzzyGoal::CriterionMembership

Description

Stores the three membership values of one criterion evaluation.

Member	Description
`double desirable`	Desirable membership value.
`double tolerable`	Tolerable membership value.
`double undesirable`	Undesirable membership value.

Notes

All three membership values are clamped to the interval $[0, 1]$.

10.3.10 `FuzzyGoal::RuleMembership`

enum FuzzyGoal::RuleMembership

Description

Specifies which membership class of a criterion is used in a fuzzy rule, or which output membership receives the rule contribution.

Value	Description
`Desirable`	Desirable membership.
`Tolerable`	Tolerable membership.
`Undesirable`	Undesirable membership.

10.3.11 `FuzzyGoal::RuleOperator`

enum FuzzyGoal::RuleOperator

Description

Specifies how the two input memberships of a fuzzy rule are combined.

Value	Description
`And`	Fuzzy conjunction. The exact numerical operation depends on the selected aggregation operator.
`Or`	Fuzzy disjunction. The exact numerical operation depends on the selected aggregation operator.

10.3.12 FuzzyGoal::AggregationOperator

enum FuzzyGoal::AggregationOperator

Description

Specifies how two rule memberships are numerically aggregated.

Value	Description
`MinMax`	Uses minimum for `And` and maximum for `Or`.
`Product`	Uses product for `And` and probabilistic sum for `Or`.
`SoftMinMax`	Uses a smooth approximation of minimum or maximum. Requires a positive parameter.
`PNorm`	Uses a p-norm based approximation. Requires a negative parameter.

10.3.13 FuzzyGoal::Aggregation

struct FuzzyGoal::Aggregation

Description

Describes the aggregation operator used for a fuzzy rule. Objects of this type are created using static helper functions.

Member	Description
`AggregationOperator type`	Selected aggregation operator.
`double parameter`	Parameter used by `SoftMinMax` or `PNorm`. For `MinMax` and `Product`, this value is not used.

FuzzyGoal::Aggregation::MinMaxAgg

Signature

```
static Aggregation MinMaxAgg();
```

Description

Creates a min/max aggregation descriptor.

Returns

An `Aggregation` object using `MinMax`.

FuzzyGoal::Aggregation::ProductAgg

Signature

```
static Aggregation ProductAgg();
```

Description

Creates a product aggregation descriptor.

Returns

An `Aggregation` object using `Product`.

FuzzyGoal::Aggregation::SoftMinMaxAgg

Signature

```
static Aggregation SoftMinMaxAgg(double eps);
```

Description

Creates a smooth min/max aggregation descriptor.

Parameters

`eps`	Smoothing parameter. Must be positive.

Returns

An `Aggregation` object using `SoftMinMax`.

FuzzyGoal::Aggregation::PNormAgg

Signature

```
static Aggregation PNormAgg(double p);
```

Description

Creates a p-norm aggregation descriptor.

Parameters

`p`	P-norm parameter. Must be negative.

Returns

An `Aggregation` object using `PNorm`.

10.4 Public Methods

10.4.1 Construction

FuzzyGoal::FuzzyGoal

Signature

```
FuzzyGoal();
```

Description

Constructs an empty fuzzy objective function. Initially, no criteria and no rules are registered.

10.4.2 Status Conversion

FuzzyGoal::statusToString

Signature

```
static const char* statusToString(
       AddCriterionStatus status);
static const char* statusToString(
       AddRuleStatus status);
static const char* statusToString(
       EvaluationStatus status);
```

Description

Converts a status code to a human-readable string. Overloads are provided for criterion creation, rule creation, and evaluation status codes.

Parameters

`status`	Status code to convert.

Returns

A string representation of the status code.

Example

```
auto rule = goal.addRule(...);

if (rule.status != FuzzyGoal::RuleOk)
{
    std::cerr << FuzzyGoal::statusToString(rule.status) << "\n";
}
```

10.4.3 Criterion Creation

FuzzyGoal::addCriterion

Signature

```
NewCriterion addCriterion(
             const std::string& name,
             double c0, double c1, double cRef,
             Direction direction,
             MembershipFunction mf);
```

Description

Adds a criterion using the default boundary behavior `HardSaturation`. If a Gaussian membership function is selected, the default value `sigmaRel = 0.15` is used.

Parameters

`name`	Name of the criterion. The name is stored by the object and is mainly used for user-side identification.
`c0`	Lower interval boundary.
`c1`	Upper interval boundary. Must be larger than `c0`.
`cRef`	Reference value. Must lie strictly inside the interval.
`direction`	Criterion direction, either `SmallerIsBetter` or `LargerIsBetter`.
`mf`	Membership function variant.

Returns

A `NewCriterion` object containing a status code and, if successful, the criterion ID.

Notes

The returned ID is valid only if `status == FuzzyGoal::CriterionOk`.

FuzzyGoal::addCriterion

Signature

```
NewCriterion addCriterion(
             const std::string& name,
             double c0, double c1, double cRef,
             Direction direction,
             MembershipFunction mf,
             double sigmaRel);
```

Description

Adds a criterion with an explicit `sigmaRel` value. This overload is mainly relevant for Gaussian membership function variants.

Parameters

`name`	Name of the criterion.
`c0`	Lower interval boundary.
`c1`	Upper interval boundary.
`cRef`	Reference value.
`direction`	Criterion direction.
`mf`	Membership function variant.
`sigmaRel`	Relative Gaussian width parameter. Must be positive for Gaussian variants.

Returns

A `NewCriterion` object.

FuzzyGoal::addCriterion

Signature

```
NewCriterion addCriterion(
             const std::string& name,
             double c0, double c1, double cRef,
             Direction direction,
             MembershipFunction mf,
             BoundaryBehavior boundaryBehavior);
```

Description

Adds a criterion with explicitly selected boundary behavior. If a Gaussian membership function is selected, the default value `sigmaRel = 0.15` is used.

Parameters

`name`	Name of the criterion.
`c0`	Lower interval boundary.
`c1`	Upper interval boundary.
`cRef`	Reference value.
`direction`	Criterion direction.
`mf`	Membership function variant.
`boundaryBehavior`	Behavior outside the modeled interval.

Returns

A `NewCriterion` object.

FuzzyGoal::addCriterion

Signature

```
NewCriterion addCriterion(
             const std::string& name,
             double c0, double c1, double cRef,
             Direction direction,
             MembershipFunction mf,
             BoundaryBehavior boundaryBehavior,
             double sigmaRel);
```

Description

Adds a criterion with explicitly selected boundary behavior and explicitly specified `sigmaRel`.

Parameters

`name`	Name of the criterion.
`c0`	Lower interval boundary.
`c1`	Upper interval boundary.
`cRef`	Reference value.
`direction`	Criterion direction.
`mf`	Membership function variant.
`boundaryBehavior`	Behavior outside the modeled interval.
`sigmaRel`	Relative Gaussian width parameter.

Returns

A `NewCriterion` object.

Notes

For all ordinary criteria, the following condition must hold: $c_0 < c_{\text{Ref}} < c_1$.

10.4.4 Equality Constraints

FuzzyGoal::addEqualityConstraint

Signature

```
NewCriterion addEqualityConstraint(
             const std::string& name,
             double targetValue,
             double tolerance,
             MembershipFunction mf = GaussianGlobalGaussianTol,
             double sigmaRel = 0.15);
```

Description

Adds an equality constraint with target value `targetValue`. The user passes the raw value $g(x)$ during evaluation. Internally, the library evaluates the absolute deviation $c = |g(x) - g^*|$.

Parameters

`name`	Name of the equality constraint.
`targetValue`	Target value g^*.
`tolerance`	Tolerance value. Internally used as the reference value of the deviation criterion. Must be positive.
`mf`	Membership function variant. Defaults to `GaussianGlobalGaussianTol`.
`sigmaRel`	Relative Gaussian width parameter. Defaults to `0.15`.

Returns

A `NewCriterion` object.

FuzzyGoal::addEqualityConstraint

Signature

```
NewCriterion addEqualityConstraint(
             const std::string& name,
             double targetValue,
             double tolerance,
             MembershipFunction mf,
             double sigmaRel,
             double gamma);
```

Description

Adds an equality constraint with explicit `gamma`. Internally, the deviation criterion is constructed as:
$c_0 = 0, \qquad c_{\text{Ref}} = \text{tolerance}, \qquad c_1 = \gamma \cdot \text{tolerance}.$

Parameters

`name`	Name of the equality constraint.
`targetValue`	Target value g^*.
`tolerance`	Tolerance value. Must be positive.
`mf`	Membership function variant.
`sigmaRel`	Relative Gaussian width parameter.
`gamma`	Factor defining the upper interval boundary. Must be greater than one.

Returns

A `NewCriterion` object.

FuzzyGoal::addEqualityConstraint

Signature

```
NewCriterion addEqualityConstraint(
             const std::string& name,
             double tolerance,
             MembershipFunction mf = GaussianGlobalGaussianTol,
             double sigmaRel = 0.15);
```

Description

Convenience overload for equality constraints with target value zero.
This overload is equivalent to calling the target-value version with `targetValue = 0.0`.

Parameters

`name`	Name of the equality constraint.
`tolerance`	Tolerance value. Must be positive.
`mf`	Membership function variant. Defaults to `GaussianGlobalGaussianTol`.
`sigmaRel`	Relative Gaussian width parameter. Defaults to `0.15`.

Returns

A `NewCriterion` object.

FuzzyGoal::addEqualityConstraint

Signature

```
NewCriterion addEqualityConstraint(
             const std::string& name,
             double tolerance,
             MembershipFunction mf,
             double sigmaRel,
             double gamma);
```

Description

Convenience overload for equality constraints with target value zero and explicit `gamma`.

Parameters

`name`	Name of the equality constraint.
`tolerance`	Tolerance value. Must be positive.
`mf`	Membership function variant.
`sigmaRel`	Relative Gaussian width parameter.
`gamma`	Factor defining the upper interval boundary. Must be greater than one.

Returns

A `NewCriterion` object.

10.4.5 Criterion Removal

FuzzyGoal::removeCriterion

Signature

```
void removeCriterion(int criterionId);
```

Description

Removes the criterion with the given ID.

Parameters

`criterionId`	ID of the criterion to remove.

Notes

Rules that refer to the removed criterion are removed automatically.

10.4.6 Rule Creation

FuzzyGoal::addRule

Signature

```
NewRule addRule(
        int criterionA, RuleMembership memA,
        int criterionB, RuleMembership memB,
        RuleOperator op,
        RuleMembership outcome);
```

Description

Adds a fuzzy rule using the default min/max aggregation.
The rule combines one membership of criterion A with one membership of criterion B and adds the resulting contribution to the selected output membership.

Parameters

`criterionA`	ID of the first criterion.
`memA`	Membership class selected from the first criterion.
`criterionB`	ID of the second criterion.
`memB`	Membership class selected from the second criterion.
`op`	Logical rule operator, either `And` or `Or`.
`outcome`	Output membership receiving the rule contribution.

Returns

A `NewRule` object containing a status code and, if successful, the rule ID.

Notes

The returned ID is valid only if `status == FuzzyGoal::RuleOk`.

FuzzyGoal::addRule

Signature

```
NewRule addRule(
        int criterionA, RuleMembership memA,
        int criterionB, RuleMembership memB,
        RuleOperator op,
        RuleMembership outcome,
        Aggregation aggregation);
```

Description

Adds a fuzzy rule using an explicitly selected aggregation operator.

Parameters

`criterionA`	ID of the first criterion.
`memA`	Membership class selected from the first criterion.
`criterionB`	ID of the second criterion.
`memB`	Membership class selected from the second criterion.
`op`	Logical rule operator.
`outcome`	Output membership receiving the rule contribution.
`aggregation`	Aggregation descriptor used to combine the two selected memberships.

Returns

A `NewRule` object containing a status code and, if successful, the rule ID.

Notes

This method returns `RuleInvalidAggregation` if the aggregation parameters are invalid. For `SoftMinMax`, the aggregation parameter must be positive. For `PNorm`, the aggregation parameter must be negative.

10.4.7 Rule Removal

FuzzyGoal::removeRule

Signature

```
void removeRule(int ruleId);
```

Description

Removes the rule with the given ID.

Parameters

`ruleId`	ID of the rule to remove.

10.4.8 Criterion Evaluation

FuzzyGoal::evaluateCriterionMembership

Signature

```
EvaluationStatus evaluateCriterionMembership(
                 int criterionId,
                 double value,
                 CriterionMembership& membership) const;
```

Description

Evaluates the membership values of a single criterion.
The resulting desirable, tolerable, and undesirable memberships are written to the `membership` output parameter.

Parameters

`criterionId`	ID of the criterion to evaluate.
`value`	Criterion value. For equality constraints, this is the raw value $g(x)$; the absolute deviation from the target value is computed internally.
`membership`	Output parameter receiving the three membership values.

Returns

`EvaluationOk` if the criterion was found and evaluated successfully. Returns `EvaluationUnknownCriterion` if the criterion ID is unknown.

Notes

In typical applications, users do not need to call this method explicitly. The membership values are computed internally when `evaluate` is called. This method is mainly useful for debugging, inspection, and plotting.

10.4.9 Objective Evaluation

FuzzyGoal::evaluate

Signature

```
EvaluationStatus evaluate(
                 const std::vector<std::pair<int,double>>& values,
                 double& objectiveValue) const;
```

Description

Evaluates the complete fuzzy objective function and computes the scalar objective value. The input vector contains pairs of criterion IDs and corresponding criterion values. The evaluation is strict: every registered criterion must be provided exactly once.

Parameters

`values`	Vector of criterion ID/value pairs. Each registered criterion must occur exactly once.
`objectiveValue`	Output parameter receiving the scalar objective value. This value is valid only if the returned status is `EvaluationOk`.

Returns

An `EvaluationStatus` value describing whether the evaluation was successful.

Notes

The method returns:

- `EvaluationOk` if the objective value was computed successfully,
- `EvaluationUnknownCriterion` if an unknown criterion ID is provided,
- `EvaluationMissingCriterion` if a registered criterion is missing,
- `EvaluationDuplicateCriterion` if a criterion occurs more than once,
- `EvaluationDefuzzificationFailed` if no valid defuzzification is possible.

If successful, `objectiveValue` lies in the interval $[0, 1]$. Smaller values indicate a more desirable result.

Example

```
double objective = 0.0;

auto status = goal.evaluate(
{
    {cost.id, 40.0},
    {accuracy.id, 0.82}
},
objective
);

if (status != FuzzyGoal::EvaluationOk)
{
    std::cerr << "Evaluation failed: "
    << FuzzyGoal::statusToString(status) << "\n";
}
```

11 Recommendations

- Always check the status returned by `addCriterion`, `addEqualityConstraint`, `addRule`, and `evaluate`.
- Use `statusToString` to print readable error messages.
- Store the criterion IDs returned during criterion creation. They are required for rule creation and evaluation.
- Store rule IDs if rules need to be removed later.
- Use `evaluateCriterionMembership` only when individual membership values are needed for debugging, inspection, or plotting.
- Use `evaluate` to compute the final scalar objective value.
- For Gaussian membership functions, ensure that `sigmaRel` is positive.
- For `SoftMinMax`, use a positive parameter.
- For `PNorm`, use a negative parameter.
- Remember that `evaluate` requires exactly one value for every registered criterion.

References

[1] O. Frommann, *Objective Functions in Multi-Criteria Optimization: Weighting, Fuzzy Logic, and Solution-Space Topography*, 2026. doi: will be added once available.